COMPUTER PROGRAMMING
LAB 8
Lab Title: Two-Dimensional Arrays in C++

1. OBJECTIVE

The aims of this laboratory experiment were:

- To study the working principle of two-dimensional arrays in C++.
- To practice creating and initializing arrays in matrix form.
- To understand how nested loops are used to access rows and columns.
- To perform different operations on 2D arrays such as calculating sums and locating the largest value.
- To improve programming skills related to matrix-based data handling.

2. THEORY

Introduction to Two-Dimensional Arrays

A two-dimensional array is a collection of elements arranged in rows and columns. It is similar to a table or matrix where each value is identified by two positions: one for the row and one for the column.

2D arrays are useful when handling structured data such as marksheets, matrices, seating charts, and image pixels.

Declaring a 2D Array

To declare a two-dimensional array, both row size and column size must be specified.

Syntax

dataType arrayName[rowSize][columnSize];

Example

int marks[3][4];

This creates an integer array having 3 rows and 4 columns.

Initializing a 2D Array

Values can be assigned during declaration by using braces for each row.

```
int table[2][3] =
{
```

```
    {1, 2, 3},
    {4, 5, 6}
};
```

Accessing Elements

Each element is accessed using row and column indexes.

table[1][2];

Nested loops are commonly used to process all elements of a matrix.

3. SOFTWARE USED

- DEV-C++ Programming Environment
- GNU C++ Compiler

4. TASKS

Task 1 — Creating and Displaying a Matrix

A two-dimensional array named table of size 3×3 was initialized with integer values. Nested loops were used to print the matrix row-wise in tabular form.

```
#include <iostream>
using namespace std;

int main()
{
    int table[3][3] =
    {
        {9, 8, 7},
        {6, 5, 4},
        {3, 2, 1}
    };

    for(int row = 0; row < 3; row++)
    {
        for(int col = 0; col < 3; col++)
        {
            cout << table[row][col] << "\t";
        }

        cout << endl;
    }

    return 0;
}
```

Task 2 — Calculating Row and Column Totals

A 3×3 matrix was created. The program calculates the sum of every row and every column separately. Finally, the complete sum of all matrix elements is displayed.

```
#include <iostream>
using namespace std;

int main()
{
   int numbers[3][3] =
   {
      {5, 1, 3},
      {2, 7, 4},
      {6, 8, 9}
   };

   int grandTotal = 0;

   // row sums
   for(int r = 0; r < 3; r++)
   {
      int sumRow = 0;

      for(int c = 0; c < 3; c++)
      {
         sumRow += numbers[r][c];
      }

      cout << "Sum of row "
           << r
           << " = "
           << sumRow
           << endl;

      grandTotal += sumRow;
   }

   // column sums
   for(int c = 0; c < 3; c++)
   {
      int sumCol = 0;

      for(int r = 0; r < 3; r++)
      {
         sumCol += numbers[r][c];
      }

      cout << "Sum of column "
           << c
           << " = "
           << sumCol
```

```
            << endl;
    }

    cout << "Grand Total = " << grandTotal << endl;

    return 0;
}
```

Task 3 — Searching Maximum Element in Matrix

A 4×4 matrix containing integer values was declared. The program compares all elements one by one to determine the largest number present in the array.

```
#include <iostream>
using namespace std;

int main()
{
    int data[4][4] =
    {
        {10, 22, 13, 4},
        {7, 35, 18, 2},
        {41, 9, 16, 25},
        {12, 5, 30, 8}
    };

    int largest = data[0][0];

    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 4; j++)
        {
            if(data[i][j] > largest)
            {
                largest = data[i][j];
            }
        }
    }

    cout << "Largest element = " << largest << endl;

    return 0;
}
```

5. CONCLUSION

This lab enhanced the understanding of two-dimensional arrays and matrix operations in C++. Different tasks including matrix display, row and column calculations, and searching for the maximum element were completed successfully. The use of nested loops with 2D arrays demonstrated how matrix-style data can be processed efficiently in programming applications.